The Link-State (LS) Routing Algorithm

1 Overview: Link-State (LS) Approach

A Link-State (LS) algorithm requires a **global view** of the network. This is achieved through a process called **Link-State Broadcast**.

How LS Obtains Global Knowledge

The Mechanism: Link-State Broadcast (Flooding)

- **Link-State Advertisements (LSAs):** Each node creates a packet containing:
  - Its own identity (Node ID).
  - List of **all neighbors** and the **cost** to reach each.
  - **Sequence Number:** To distinguish between new and old information.
  - **Time-to-Live (TTL):** To discard stale packets.
- **Reliable Flooding:** When a router receives an LSA:
  - It stores the packet in its database.
  - It forwards the packet to **all neighbors** (except the one it received it from).
- **The Result:** Eventually, every node learns about every other node's surroundings, resulting in an **identical Link-State Database (LSDB)**.

Visualizing the Global Knowledge Process

Suppose we have a simple triangle network **A** — **B** — **C** — **A**.

1. **Discovery:** Node **A** detects it has neighbors **B** (cost 10) and **C** (cost 5).
2. **Generation:** Node **A** generates an LSA: {`Src: A, Neighbors: (B:10), (C:5), Seq: 001`}.
3. **Flooding:** Node **A** sends this LSA to **B** and **C**.
4. **Convergence:** **B** receives A's info and forwards it to **C**. **C** does the same. Now, **B** and **C** both know exactly who A's neighbors are and the costs to reach them.

Once this happens for ALL nodes, everyone can draw the full map of the network individually.

2 Dijkstra's Algorithm

The most famous LS algorithm is **Dijkstra's Algorithm**. It is an iterative algorithm that computes the least-cost path from a **source node (u)** to all other nodes.

2.1 Basic Notation

Notation for Dijkstra's

- $D(v)$**:** Cost of the least-cost path from source to destination v as of the current iteration.
- $p(v)$**:** Predecessor node (neighbor of v) along the current least-cost path.
- N'**:** Subset of nodes whose least-cost path from the source is **definitively known**.

2.2 The Algorithm (Pseudocode)

Dijkstra's LS Algorithm for Source u

1. **Initialization:**
   - $N' = \{u\}$
   - For all nodes v:
     - If v is neighbor of u, then $D(v) = c(u, v)$
     - Else $D(v) = \infty$
2. **Loop:**
   - Find w not in N' such that $D(w)$ is a minimum.
   - Add w to N'.
   - **Update** $D(v)$ for each neighbor v of w that is NOT in N':

$$D(v) = \min(D(v), D(w) + c(w, v))$$

3. **Termination:** Repeat until $N' = N$.

3 Example Trace

Let's trace the algorithm on the network from Figure 5.3, starting from source node u.

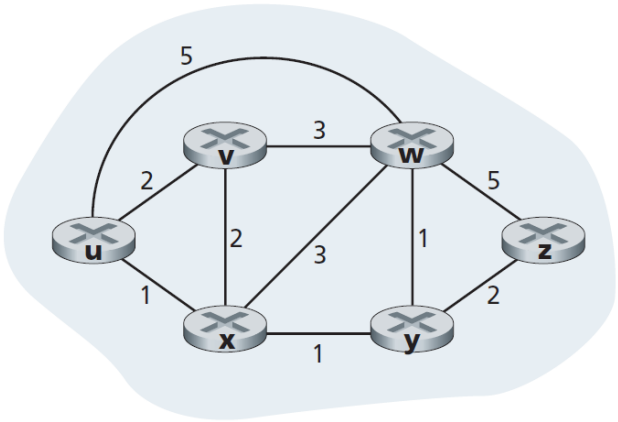

Figure 1: The sample network used for the Dijkstra's algorithm trace

Step	N'	$D(v), p(v)$	$D(w), p(w)$	$D(x), p(x)$	$D(y), p(y)$	$D(z), p(z)$
0	u	2, u	5, u	1, u	∞	∞
1	ux	2, u	4, x	–	2, x	∞
2	uxy	2, u	3, y	–	–	4, y
3	$uxyv$	–	3, y	–	–	4, y
4	$uxyvw$	–	–	–	–	4, y
5	$uxyvwz$	–	–	–	–	–

Table 1: Step-by-step execution of Dijkstra's Algorithm from source u. *Note: Ties in costs (e.g., step 2) can be broken arbitrarily.*

3.1 Path Reconstruction & Forwarding Table

How to find the Full Path?

For any node v, we look at $p(v)$ to find the previous node. We then look at $p(p(v))$, and so on, until we reach source u. This constructs the path in reverse $(v \leftarrow p(v) \leftarrow \cdots \leftarrow u)$.

Resulting Forwarding Table in Node u

For each destination, the forwarding table only needs the **next-hop** node (the first node on the least-cost path).

Destination	Outgoing Link (Next Hop)
v	(u, v)
w	(u, x)
x	(u, x)
y	(u, x)
z	(u, x)

4 Algorithm Analysis

4.1 Computational Complexity

- **Search:** In each iteration, we search through $n, n-1, \ldots, 1$ nodes to find the minimum $D(w)$.
- **Total Complexity:** $\sum n + (n-1) + \cdots + 1 = \frac{n(n+1)}{2} \approx \mathbf{O(n^2)}$.
- **Optimization:** Using a **Heap** data structure can reduce the minimum-finding step to logarithmic time, achieving $\mathbf{O(E \log V)}$.

4.2 Pathologies: Routing Oscillations

Oscillations can occur when link costs depend on **traffic load** (e.g., delay).

The Oscillation Problem

- **Cause:** Routers simultaneously shift traffic to "low-cost" paths, causing them to become congested, then shift back.
- **Self-Synchronization:** Routers can "self-synchronize", running the algorithm at the same time and causing mass oscillations.
- **Solution 1:** Keep link costs independent of load (common in OSPF).
- **Solution 2: Randomize** when each router sends advertisements to break synchronization.